

粒子群算法求解最大值问题详解（附python代码）

文章目录粒子群算法（PSO）代码实现1、导入需要的库2、设置参数3、适应度函数4、初始化粒子群5、迭代更新粒子群
粒子群算法（PSO）

PSO 通过模拟鸟群的捕食行为来求取最优解。

假设一群鸟在随机搜索食物。在这个区域里只有一块食物（对应着最优解）。所有的鸟都不知道食物的具体位置，但是它们可以判断自身与食物的大致距离，即通过 **fit** 值判断与最优解的距离。那么找到食物的最优策略就是搜寻目前离食物最近的鸟的周围区域。

PSO 中，问题的每个解都是搜索空间中的一只“鸟”。我们称之为“粒子”。所有的粒子都有一个由被优化的函数决定的适应值，并且所有的粒子都具有速度和位置两个属性。

在每一次迭代中，粒子通过跟踪两个“极值”来更新自己。第一个极值就是粒子本身所找到的最优解 **p_best**。另一个极值是整个种群目前找到的最优解，即全局极值 **g_best**。

粒子群算法的求解伪代码如下：

```
初始化粒子群
while 未达到最大迭代次数或最小损失：
    for 粒子 in 粒子群:
        计算粒子适应度
        if 此粒子适应度高于此粒子在所有迭代次数的最佳适应度 p_bestfit:
            将该值设置为新的 p_bestfit
        if 所有粒子中的最佳适应度大于整个粒子群在所有迭代次数的最佳适应度 p_bestfit:
            选择有最佳适应度的粒子作为 g_best，将该适应度设置为新的 g_bestfit
    for 粒子 in 粒子群:
        计算粒子速度
        更新粒子位置
```

其中，粒子速度的更新公式为：

$$v[i] = w * v[i] + c1 * rand() * (pbest[i] - present[i]) + c2 * rand() * (gbest - present[i])$$

其中：

$$c1 * rand() * (pbest[i] - present[i])$$

代表利用粒子本身所找到的最优解更新自己的速度。

$$c2 * rand() * (gbest - present[i])$$

代表利用整个种群目前找到的最优解更新自己的速度。

粒子位置的更新公式为：

$$present[i] = present[i] + v[i]$$

在以上公式中， $v[i]$ 是第 i 个粒子的速度， w 是惯性权重（有助于跳出局部最优解）， $present[i]$ 是第 i 个粒子的当前位置， $pbest[i]$ 是第 i 个粒子的历史最佳， $gbest$ 是全局最佳， $rand()$ 是介于 0 到 1 之间的随机数。 $c1$ 、 $c2$ 是学习因子，通常情况下， $c1=c2=2$ 。

代码实现

1、导入需要的库

```
import numpy as np
```

2、设置参数

```
w = 0.8 #惯性因子
c1 = 2 #自身认知因子
c2 = 2 #社会认知因子
r1 = 0.6 #自身认知学习率
r2 = 0.3 #社会认知学习率
pN = 50 #粒子数量
dim = 1 #搜索维度
max_iter = 300 #最大迭代次数
```

3、适应度函数

```
def get_fitness(x):
    y = -x**2 + 20*x + 10
    return y
```

4、初始化粒子群

```
X = np.zeros((pN, dim)) #初始粒子的位置和速度
V = np.zeros((pN, dim))
p_best = np.zeros((pN, dim), dtype = float) #粒子历史最佳位置
g_best = np.zeros((1, dim), dtype = float) #全局最佳位置
p_bestfit = np.zeros(pN) #每个个体的历史最佳适应值
g_bestfit = -1e15 #全局最佳适应值
```

```
for i in range(pN):
    #初始化每一个粒子的位置和速度
    X[i] = np.random.uniform(0,5,[1, dim])
    V[i] = np.random.uniform(0,5,[1, dim])
    p_best[i] = X[i] #初始化历史最佳位置
```

```
p_bestfit[i] = get_fitness(X[i]) #得到对应的fit值
if p_bestfit[i] > g_bestfit:
g_bestfit = p_bestfit[i] g_best = X[i] #得到全局最佳
```

5、迭代更新粒子群

```
fitness = []

for _ in range(max_iter):
for i in range(pN): #更新g_best\p_best
temp = get_fitness(X[i]) #获得当前位置的适应值
if temp > p_bestfit[i]: #更新个体最优
p_bestfit[i] = temp
p_best[i] = X[i] if p_bestfit[i] > g_bestfit: #更新全局最优
g_best = X[i] g_bestfit = p_bestfit[i]

for i in range(pN): #更新权重
V[i] = w*V[i] + c1*r1*(p_best[i] - X[i]) + c2*r2*(g_best - X[i])
X[i] = X[i] + V[i]
```

```
fitness.append(g_bestfit)
```

```
print(g_best, g_bestfit)
```

作者：cofisher